

Solução de Inteligência na Borda com AWS Greengrass V2 para Reconhecimento de Veículos usando YOLOv8

1st Matheus Alves

Laboratório de Comunicações e Sinais (LCS)
Escola Politécnica da Universidade de São Paulo (Poli-USP)
São Paulo, Brasil
mfalves@lcs.poli.usp

Abstract—Este trabalho apresenta a implementação de uma solução de computação em borda utilizando AWS Greengrass V2 para detecção de veículos com o modelo YOLOv8. O objetivo foi demonstrar a viabilidade de processar dados de imagem localmente em um dispositivo configurado como Greengrass Device, minimizando a latência e o tráfego de dados para a nuvem. O projeto foi desenvolvido em ambiente WSL (*Windows Subsystem for Linux*), com integração ao AWS IoT Core para comunicação segura e publicação de resultados via MQTT. Os resultados obtidos mostram que a inferência em borda proporciona maior eficiência no uso da rede, mantendo acurácia adequada para aplicações de visão computacional, atingindo uma latência mínima de 250ms na detecção das classes de veículos configuradas pelo modelo. As mensagens MQTT com os resultados de inferência enviadas pelo dispositivo Greengrass a AWS foram menores que 1.6KB, uma redução significativa em relação aos modelos tradicionais de computação em nuvem.

Index Terms—AWS Greengrass V2, YOLO, Computação de Borda, IoT, ITS

I. INTRODUÇÃO

Atualmente, sistemas de transporte são uma parte indispensável da atividade humana [1]. De acordo com [1], 94% dos acidentes em estradas e rodovias são causados por erros humanos. Além disso, outra pesquisa mostra que os acidentes são responsáveis por 50%-60% dos congestionamentos nas grandes cidades [2].

Neste sentido, o uso de Sistemas de Transporte Inteligente (ITS - *Intelligent Transportation Systems*) surgiram como uma forma eficiente de melhorar a performance dos sistemas de transporte [2]. A ideia do uso de ITSs não é nova. Em 2011, artigos já citavam o potencial revolucionário dessa aplicação e seus diversos benefícios, tais como: redução no número de acidentes de trânsito, diminuição de engarrafamentos, atenuação da emissão de gases do efeito estufa, entre outros [2].

Uma das principais aplicações que ITSs são capazes de fornecer é a detecção de veículos em estradas/rodovias utilizando imagens enviadas por dispositivos IoT (*Internet of Things*), tais como câmeras de vigilância, de forma automatizada, a partir de algoritmos de processamento de imagens, que podem ser utilizadas para gerenciamento do tráfego de veículos, constatação de acidentes, rastreamento de veículos em fuga, entre outros [1], [2].

Uma das maneiras de implementar esse tipo de serviço é enviando os dados capturados (e.g., vídeo produzido pela câmera de vigilância) para um sistema de gerenciamento com alta capacidade computacional, como por exemplo aqueles alocados em ambientes de nuvem como a AWS (*Amazon Web Service*), que faz o tratamento, informa atividades relevantes e toma decisões baseada em regras pré-estabelecidas [3].

Por mais que essa solução apresente vantagens do ponto de vista da alta capacidade de processamento, ela também apresenta uma série de limitações em relação aos requisitos para a aplicação dos ITSs de forma eficiente e segura, tais como:

- 1) Necessidade de processamento em tempo real (baixa latência).
- 2) Questões relacionadas a privacidade dos dados coletados.
- 3) Custo para implementação em larga escala (devido, por exemplo, a alta largura de banda necessária para transmitir vídeos em alta resolução para a nuvem).

Considerando os requisitos citados acima, o uso de tecnologias de Computação em Borda (EC - *Edge Computing*) mostra-se mais adequado. Na EC, o processamento de dados é aproximado da sua fonte de origem (a "borda" da rede). Como exemplo, ao invés de enviar um *stream* de vídeo bruto para a nuvem para análise, as imagens são processadas localmente e apenas os resultados são enviados (e.g., "carro detectado às 15:32:10"). Essa abordagem reduz a latência do processo, economiza banda e permite operações mesmo com conectividade intermitente [3].

Neste sentido, o objetivo deste trabalho é apresentar uma solução baseada em computação em borda para detecção de veículos que permite a implementação de ITSs em larga escala. Para isso, foi utilizado o algoritmo de Aprendizagem de Máquina (ML - *Machine Learning*) para detecção e classificação de objetos YOLOv8, que pertence a família de algoritmos *You Only Look Once*, popular nesse tipo de aplicação.

O algoritmo YOLOv8 da Ultralytics, rodando em um *script python* instalado em um Notebook *Dell Inspiron 5458* em ambiente WSL (*Windows Subsystem for Linux*) tem como

entrada as imagens de uma câmera de vigilância / radar (emulado por um vídeo rodando em *loop*). Após realizar a inferência, os dados são transmitidos para a AWS utilizando o serviço *AWS Greengrass V2*.

Através do *Greengrass Core*, instalado num ambiente WSL local, o notebook é transformado em um dispositivo IoT, que se comunica com o serviço AWS IoT Core para enviar mensagens à nuvem AWS usando o protocolo MQTT (*Message Queuing Telemetry Transport*).

As principais contribuições fornecidas neste artigo são:

- Demonstra a viabilidade do desenvolvimento de uma aplicação de detecção e rastreamento de veículos em tempo real utilizando a tecnologia AWS Greengrass V2 + MQTT com baixo custo, alta escalabilidade e baixa latência.
- Mostra como o uso da EC reduz a banda necessária para implementar soluções IoT em nuvem.
- Exibe como a integração de diferentes tecnologias como ML, IoT, computação em nuvem, entre outras pode ser utilizada na solução de problemas práticos da sociedade.

O resto deste artigo está dividido da seguinte forma: A Seção II exibe a base teórica necessária para compreender os conceitos por trás da metodologia utilizada na criação da solução proposta.

A Seção III discute como foi feita a implementação da solução, descrevendo o *Setup* experimental, configuração de ambiente, *softwares* e bibliotecas utilizadas no projeto.

Além disso, descreve em detalhes o algoritmo utilizado para realizar a inferência dos objetos no vídeo fornecido como entrada para o YOLOv8, a integração com o AWS IoT Core e o envio dos resultados para a AWS via MQTT.

Já a Seção IV demonstra os resultados obtidos após a implementação da solução de borda, exibindo como a AWS recebe os dados no tópico MQTT configurado em formato JSON, a latência para inferência pelo YOLOv8, a taxa de detecção do algoritmo e informações relativas ao tráfego MQTT durante a comunicação entre o *Greengrass Device* e a AWS. Por fim, a Seção V conclui este trabalho.

II. FUNDAMENTAÇÃO TEÓRICA

A. Computação em Nuvem x Computação em Borda

Vários dispositivos como postes inteligentes, câmeras de segurança, sensores dos diversos tipos (como aqueles instalados em relógios inteligentes ou marcapassos, por exemplo) fazem parte de uma categoria de dispositivos denominada Internet-das-Coisas (IoT - *Internet of Things*). Dispositivos IoT tem a capacidade de sentir (ou de sensoriar, do termo *sense*), e podem produzir quantidades massivas de dados [4].

Tradicionalmente, qualquer dispositivo que gera dados para processamento, e que precisa de um poder computacional não disponível localmente, envia a informação crua para tratamento na nuvem (i.e., série de *datacenters* com alto poder computacional distante da borda da rede).

Todavia, devido a grande quantidade de dados gerada pelos dispositivos IoT, essa abordagem não é viável devido

a restrições relacionadas a largura de banda das redes de comunicação [ref].

Como exemplo, câmeras em veículos autônomos capturam grandes quantidades de dados, normalmente em forma de vídeos, tal que o sistema deve processar em tempo real. Se houver a necessidade do veículo enviar dados para a nuvem para processamento, o tempo de resposta seria muito longo. Além disso, um grande número de veículos autônomos em uma área estressaria a largura de banda da rede móvel que seria utilizada para enviar os dados [4].

A partir do exemplo anterior, é fácil inferir que o principal gargalo do modelo de computação em nuvem tradicional está relacionado com a capacidade de transferência de dados limitada entre os dispositivos finais na borda da rede e a nuvem [ref].

A Figura 1 apresenta uma comparação entre os modelos de Computação em Nuvem (CC - *Cloud Computing*) e Computação em Borda (EC - *Edge Computing*).

Em ambas as abordagens, dados do dispositivo final (e.g., dispositivo IoT) são enviados para uma entidade remota, normalmente um servidor, que tem alta capacidade de processamento e permite a inferência de resultados a partir dos dados coletados pelos dispositivos IoT.

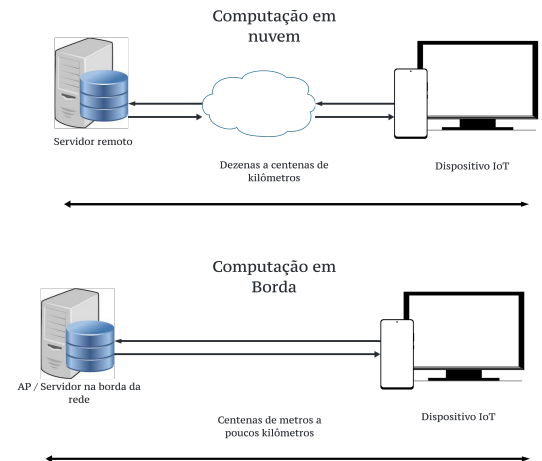


Fig. 1. Comparação de modelos de CC versus EC. Após o envio dos dados pelo dispositivo IoT, o servidor realiza o processamento e pode devolver alguma resposta. Fonte: Autor.

Todavia, conforme mostra a Figura 1, a principal diferença entre a CC e a EC deve-se pela distância entre as duas pontas do enlace. No caso da CC, a distância é de dezenas a centenas de quilômetros, o que aumenta a latência de comunicação, crucial para aplicações relacionadas a segurança, como comunicações veiculares e aplicações de saúde, por exemplo. Já no caso da EC, o componente responsável por processar os dados gerados pelos dispositivos IoT está na borda da rede, mais próximo do usuário final, de tal forma que a barreira de latência produzida pela CC é superada e o processamento é praticamente local [5].

A Tabela I compara os sistemas EC e CC e destaca suas vantagens, características e limitações.

TABLE I
COMPARAÇÃO DE SISTEMAS EC E CC.

	EC	CC
Hardware do servidor	Servidores de menor escala com recursos moderados	Servidores de maior escala com alta capacidade computacional
Localização do servidor	Na borda	Instalado em prédios dedicados, com tamanhos elevados
Implementação	Densamente implementado	Implementados por grandes empresas, e.g. Google e Amazon, em poucos locais no mundo
Distância do enlace	Pequena (dezenas a centenas de metros)	Grande (centenas a milhares de quilômetros)
Latência suportada	<dezenas de ms	>100 ms
Aplicações	Aplicações de latência crítica e altos requisitos computacionais	Aplicações tolerantes a latência e altos requisitos computacionais

B. AWS IoT

Utilizando a AWS IoT, é possível fornecer serviços de nuvem e suporte a dispositivos que podem ser utilizados para implementar soluções de IoT em larga escala. Dentre os serviços em nuvem de armazenamento e processamento de dados em larga escala que podem ser acessados por um aplicativo em um dispositivo IoT conectados a nuvem da Amazon estão o Amazon EC2 e AWS Lambda, além de serviços de banco de dados, como o Amazon DynamoDB [6].

No universo da IoT, a AWS IoT fornece serviços que suportam os dispositivos que interagem com o mundo e os dados que passam entre eles. Um dispositivo é um tipo de *hardware* que gerencia interfaces e comunicações, e podem incluir recursos de computação e armazenamento. Exemplos incluem Raspberry Pi, Arduino, assistentes de interface de voz, entre outros [6].

Uma interface é um componente que conecta um dispositivo ao mundo físico, podendo ser de entrada (permite que um usuário se comunique com um dispositivo) ou de saída (permite que um dispositivo se comunique com um usuário).

Os sensores se enquadram na primeira categoria de interfaces, medindo e detectando algo no mundo físico e convertendo para uma forma que o dispositivo entenda (e.g., sensores de temperatura, umidade, entre outros). Já os atuadores são interfaces de saída, que o dispositivo pode usar para controlar algo no mundo físico, como relés e motores de passo [7].

A comunicação entre os dispositivos e a nuvem AWS pode ser feita utilizando diversas tecnologias (e.g., WiFi, Dados celulares de banda larga e estreita, Rede de longa distância (LoRaWAN), entre outros). A Figura 2 exibe os serviços fornecidos pela AWS IoT.

C. AWS IoT Core

Dentre os serviços de controle exibidos na Figura 2, AWS IoT Core é um serviço de nuvem gerenciado que permite que dispositivos conectados interajam com segurança com aplicativos na nuvem AWS e outros dispositivos conectados à Internet. A arquitetura da AWS IoT Core é mostrado na Figura 3.

Conforme mostrado na Figura 3, as aplicações IoT interagem diretamente com os serviços em nuvem da AWS, como Amazon S3 e AWS Lambda. Usando os SDKs fornecidos pela AWS, as aplicações são instaladas em cima dos dispositivos IoT (também chamados de *IoT Things*, do termo em inglês).

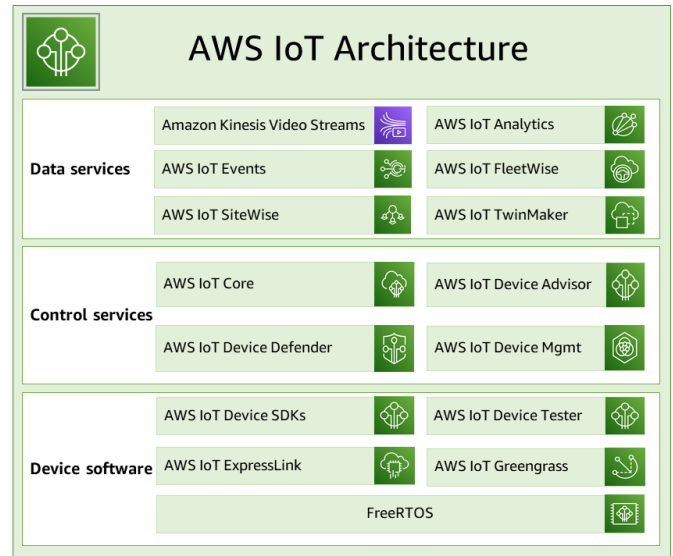


Fig. 2. Serviços fornecidos pela AWS IoT. Fonte: [7].

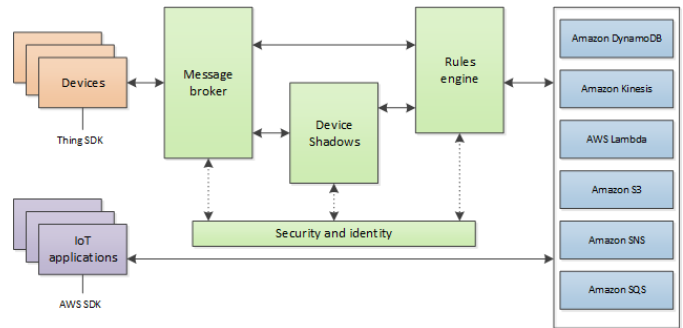


Fig. 3. Arquitetura da AWS IoT Core. Fonte: [7].

Os dispositivos IoT se comunicam com a nuvem AWS através de um operador de mensagens (do inglês, *message broker*), fornecendo um mecanismo seguro para dispositivos e aplicativos AWS IoT publicarem e receberem mensagens uns dos outros [7].

Normalmente, utiliza-se o protocolo MQTT para publicar e assinar. O *broker* distribui os dados do dispositivos por meio de tópicos para outros dispositivos e serviços do IoT Core que se inscreveram nesse tópico (mais detalhes na Seção III).

D. AWS IoT Greengrass

Considerando os conceitos de computação em borda descritos na Seção II-A, a AWS IoT fornece o serviços AWS Greengrass.

O AWS Greengrass estende o AWS IoT aos dispositivos na borda da rede, permitindo-os a agir localmente com base nos dados que geram, executar previsões com base em modelos de ML, filtrar e agregar dados do dispositivo através da instalação do seu *software* cliente no dispositivo IoT [8], [9].

Essa arquitetura permite que os dispositivos colem e analisem informações próximo à sua origem, reajam a eventos

locais em tempo real e se comuniquem de forma segura com outros dispositivos na rede local.

Adicionalmente, os dispositivos que utilizam Greengrass mantêm comunicação segura com o serviço AWS IoT Core, permitindo a exportação de dados de IoT para a nuvem da AWS quando necessário.

O desenvolvimento de aplicações de borda é realizado através de módulos de *software* pré-construídos, chamados de componentes, que podem conectar os dispositivos a serviços da AWS ou de terceiros.

Conforme mostra a Figura 4, utilizando o *software* cliente do AWS Greengrass instalado sobre um sistema operacional (como distribuições Linux e Windows), é possível habilitar a execução local de aplicações para realização de diversas tarefas utilizando funções AWS Lambda, contêineres Docker, *scripts* de várias linguagens de programação, entre outros [8], [9].

Utilizando os componentes (do inglês, *components*), pacotes de *software* pré-construídos, o AWS IoT Greengrass permite conectar os dispositivos a AWS IoT e outras aplicações na borda. Após a instalação do *software* cliente, o Greengrass permite que o usuário realize a implementação, configuração e gerenciamento das aplicações nos dispositivos IoT conectados ao Greengrass de forma remota [8], [9].

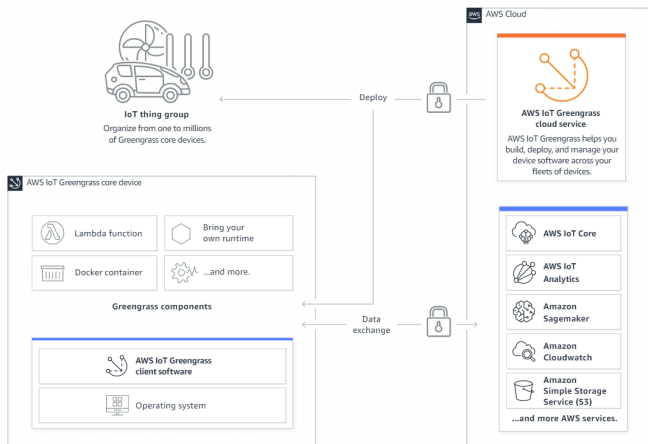


Fig. 4. Esquema de funcionamento do AWS Greengrass Fonte: [9].

Para compreender o funcionamento do AWS Greengrass, os seguintes conceitos são importantes:

- **AWS IoT Thing:** Representação de um dispositivo específico ou entidade lógica. Informações sobre uma *Thing* são armazenadas no registro AWS IoT.
- **Dispositivo Greengrass core:** Um dispositivos que executa um *software* AWS IoT Greengrass Core.
- **Dispositivo Greengrass cliente:** Um dispositivo que se conecta e comunica com o dispositivo Greengrass core através de MQTT.
- **Componente Greengrass:** Unidade fundamental de *software*, projetada para ser implantada e executada em dispositivos de borda. A arquitetura da plataforma é inteiramente baseada em componentes, permitindo que os de-

seenvolvedores utilizem módulos públicos pré-construídos ou criem suas próprias soluções customizadas.

A implantação em um ou múltiplos dispositivos é gerenciada pelo serviço em nuvem, que distribui os recursos essenciais do componente. Estes recursos incluem:

- **Receita (Recipe):** Um arquivo de configuração em formato JSON ou YAML que define os metadados do componente, seus parâmetros e ciclo de vida;
 - **Artefatos (Artifacts):** O conjunto de código-fonte, binários ou *scripts* que constituem o *software* a ser executado, podendo originar-se de funções Lambda, contêineres Docker ou runtimes customizados;
 - **Dependências (Dependencies):** O mecanismo que estabelece a relação entre componentes, garantindo que atualizações em um componente (e.g., de criptografia) acionem automaticamente a atualização e reinicialização dos componentes que dele dependem.
- **Implementação (Deploy):** Refere-se ao processo gerenciado de distribuição e aplicação da configuração de componentes a um ou mais dispositivos de destino. Uma característica central é a resolução automática de dependências, garantindo que todos os componentes necessários e suas configurações atualizadas sejam aplicados ao alvo. As implantações são contínuas, operando como uma declaração de estado desejado; qualquer modificação subsequente nos componentes ou em suas configurações é automaticamente sincronizada com todos os dispositivos vinculados.

E. Deep Learning e algoritmo YOLOv8

O Aprendizado de Máquina (DL - *Deep Learning*) é uma subárea do *Machine Learning* que permite a algoritmos realizarem previsões, classificações ou decisões baseados em dados [10].

Algoritmos de DL extraem conhecimento hierarquicamente de dados crus através de múltiplas camadas de unidades de processamento não-linear (chamadas de neurônios), com o objetivo de realizar previsões ou ações de acordo com objetivos alvo [10].

Os modelos mais conhecidos são as Redes Neurais (NN - *Neural Networks*). O principal objetivo das redes de DL, também chamadas de Redes Neurais Profundas (DNN - *Deep Neural Networks*) é aproximar funções complexas através da composição de operações simples e pré-definidas de unidades (ou neurônios) [10].

As operações executadas são geralmente definidas por uma combinação ponderada por pesos (do inglês, *weights*) de um grupo específico de neurônios ocultos com função de ativação não linear, dependendo da estrutura do modelo. Tais operações junto com as unidades de saída são chamadas de “camadas” (do inglês, *layers*) [10].

Por definição, aprendizagem profunda é a aplicação de redes multi-neurais e multicamadas para realizar tarefas de aprendizagem, incluindo regressão, classificação, *clustering*, codificação automática, entre outras [11].

A Figura 5 exibe a estrutura geral de uma DNN totalmente conectada (do inglês, *fully-connected*), na qual uma camada de entrada representando dados de entrada crus, i.e., não processados (em azul) alimenta múltiplas camadas escondidas de vários tipos que executam várias funções (em amarelo), que finalmente produzem uma saída (em branco), que pode representar valores de regressão, valores de classificação, entre outros [11].

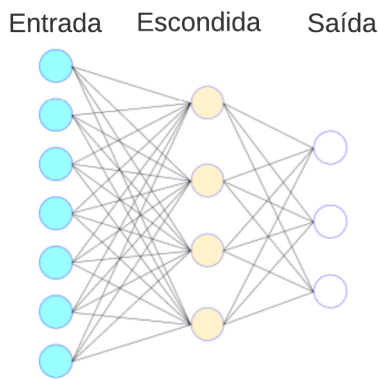


Fig. 5. Estrutura geral de uma DNN. Fonte: Adaptado de [11].

As Redes Neurais Convolucionais (CNNs - *Convolutional Neural Networks*) ganham destaque em aplicações de processamento de imagens. A 6 exibe a arquitetura de uma CNN convencional.

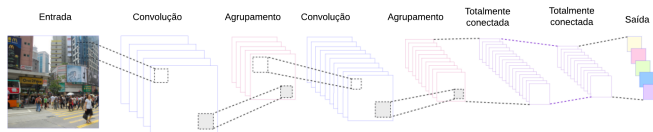


Fig. 6. Estrutura geral de uma CNN. Fonte: Adaptado de [11].

Sendo um subtipo das DNNs com foco em processamento de dados massivos (e.g., vídeos e imagens), nas CNNs camadas convolucionais e de agrupamento (do inglês, *Pooling*) são adicionadas antes das camadas *fully-connected* das estruturas de DNNs tradicionais (ver 5) [12].

F. YOLOv8

A detecção de objetos permite realizar várias tarefas úteis como rastreamento, reconhecimento de ações, previsão de posição, entre outros [13].

O problema associado com a detecção de objetos é estimar em qual parte da imagem um objeto está localizado, além de classificar qual é o tipo (ou classe) do objeto em questão [13].

Devido ao sucesso de aplicação das DNNs e CNNs, progressos foram realizados em tarefas de visão computacional como classificação, segmentação e detecção de objetos [13].

De acordo com [12], a partir do modelo YOLOv5, os modelos de detecção de objetos da família YOLO são divididos em quatro partes principais, conforme mostra a Figura 7.

- **Entrada:** Pode ser representada por uma única imagem, ou um conjunto de imagens;

- **Espinha dorsal (do inglês, *Backbone*):** Pode ser uma CNN (dentre as mais populares, destacam-se a ResNet e CSPDarkNet, etc). É projetada para fornecer a base para um detector de objetos.

Seu objetivo principal é extrair recursos (*features*) das imagens de entrada antes de submetê-las para as fases seguintes, como a fase de localização [12].

- **Pescoço (do inglês, *neck*):** Conecta o *backbone* e a cabeça, e é composta por módulos de camadas de Pooling Espacial Piramidal (SPP - *Spacial Pyramidal Pooling*) e Rede de Agregação de Caminho (PAN - *Path Aggregation Network*).

O pescoço concatena os mapas de *features* de diferentes camadas da rede de *backbone* e os envia para a entrada da cabeça [12].

- **Cabeça (do inglês, *head*):** Também é uma DNN responsável por realizar tarefas de previsão de localização e classificação (exemplos comuns são os algoritmos da família YOLO, SSD, CornerNet, entre outros) [12].

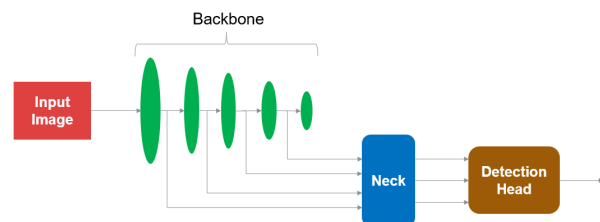


Fig. 7. Estrutura de um detector de objetos baseado em redes de DL. Fonte: Adaptado de [12].

Dentre os popularmente conhecidos detectores de objetos baseados em redes de DL, a família YOLO se destaca. Mais especificamente, o detector de objetos YOLOv8 realiza detecção e classificação de variados objetos utilizando CNNs [ref].

Antes de falar da arquitetura do YOLOv8, é necessário compreender a arquitetura do YOLOv1, o primeiro algoritmo de detecção de objetos da família YOLO. De maneira simples, ela aborda o problema da detecção de objetos em imagens como um problema de regressão único. O modelo YOLOv1 é formado por uma única NN, caixas delimitadoras (do inglês, *bounding boxes*) e a probabilidade de associação a uma certa classe em uma avaliação única [14].

O modelo base do YOLO funciona da seguinte forma: Primeiro, a imagem de entrada é dividida em um *grid* $S \times S$ onde cada célula do *grid* (i, j) realiza a predição de B *bounding boxes*, com uma nota de confiança para cada caixa, e probabilidade de pertencer a uma determinada classe C . O resultado final é um tensor de tamanho $S \times S \times (B \times 5 + C)$ [14].

A arquitetura do YOLOv1 (ver Figura 8) consiste de 24 camadas convolucionais seguido por duas camadas Totalmente Conectadas (FC - *Fully Connected*). O modelo usa o algoritmo gradiente decendente estocástico como otimizador.

Os dados poderiam ainda ser enviados via comunicação ethernet, já que normalmente esses sistemas de câmeras são conectados uma central para triagem de informações.

Vale ressaltar que, neste projeto, os dados de entrada para o *script* python que realiza a detecção de veículos foi gerada a partir de um vídeo do fluxo de veículos em uma rodovia rodando em *loop* no WSL em que o *software* Greengrass foi instalado. O vídeo pode ser encontrado em [15].

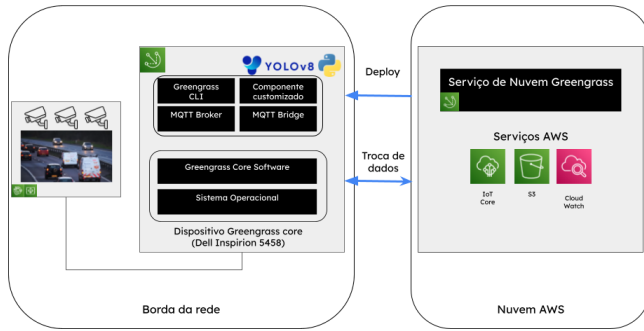


Fig. 9. Diagrama da solução de detecção de objetos usando YOLOv8 na borda da rede com AWS Greengrass V2. Fonte: Autor.

No dispositivo Greengrass *core* (ilustrado pelo notebook Dell Inspiron 5458), toda a estrutura de *software* Greengrass é instalada em cima de um sistema operacional (no caso deste projeto, Windows 10) para permitir a troca de dados e o *deploy* de *components*, permitindo executar ações na borda da rede e interagir com os serviços AWS.

O *software* Greengrass é instalado a partir do Greengrass CLI, bem como o *deploy* dos componentes no dispositivo *core*. O uso de um *Broker/Bridge* MQTT permite que o dispositivo Greengrass *core* se comunique com o serviço de nuvem Greengrass, dando acesso a serviços como o IoT Core, S3 para armazenagem de dados, entre outros.

C. Criação do Dispositivo Greengrass core e interação com AWS IoT

A Figura 10 exibe um fluxograma com o passo a passo para configurar um dispositivo Greengrass *core* e realizar o *deploy* de um componente no mesmo enquanto interage com o serviço de nuvem AWS Greengrass V2.

Utilizando o AWS CLI com usuário IAM configurado no WSL do *host*, o instalador do Greengrass *core* é baixado e executado. Utilizando o *prompt* exibido na imagem, é realizada a criação e registro da *Thing* no serviço de nuvem Greengrass, com um registro no IoT Core.

Após a criação da *thing*, um *bucket* S3 é criado para armazenagem de arquivos (como *scripts*, bibliotecas, requisitos de sistema, entre outros). Em seguida, os arquivos necessários para a execução de uma determinada tarefa de forma local é enviada para o *bucket* (como um *script* python ou arquivo Docker, por exemplo).

Uma *recipe* é então criada para envio das instruções para a implementação de um componente nos dispositivos Greengrass. No caso deste projeto, a *recipe* pode ser encontrada em [15].

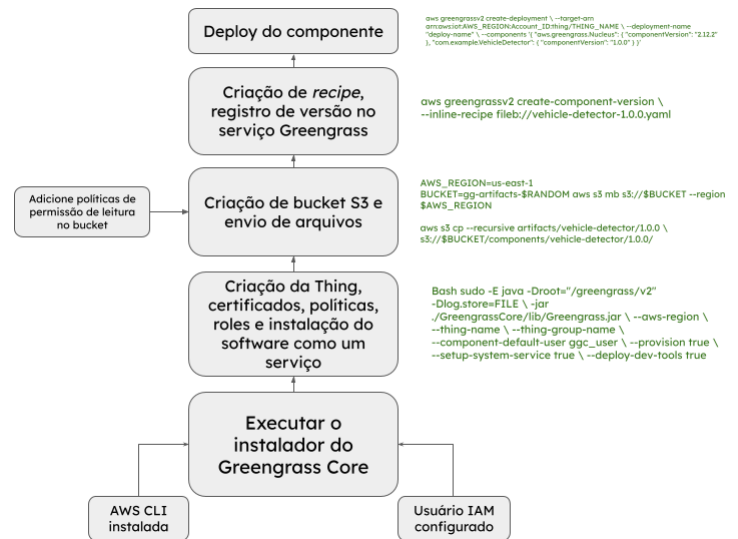


Fig. 10. Fluxograma da interação entre um dispositivo Greengrass *core* e o serviço AWS Greengrass. Fonte: Autor.

O componente foi configurado da seguinte forma:

- **Configuração padrão:** Receber um vídeo denominado "road_video.mp4", cujas detecções (configuradas com os *labels* "car,truck,bus,motorbike") e nível de confiança do detector YOLOv8 de pelo menos 0.5 são enviadas para o serviço de nuvem AWS Greengrass através do tópico "edge/vehicles/detections".
- **Políticas:** Permissão de IoT para publicação via MQTT.
- **Artefatos:** Referência a URI do *bucket* S3 contendo os arquivos necessários para a inferência local dos objetos no dispositivo Greengrass *core*.
- **Ciclo de Vida:** Criação de diretório temporário (*cache*) para armazenagem de imagens capturadas pelas câmeras. Instalação de *pip* python em ambiente *venv* (python virtual) e adição de bibliotecas necessárias para execução do *script* de inferência de objetos (ultralytics, opencv-python-headless, numpy, awsiotsdk, awscli).
- **Execução:** Execução de um *script* python que tem como entrada o vídeo "road_video.mp4", os pesos e base do modelo YOLOv8, o tópico "edge/vehicles/detections" configurado, o nível de confiança para considerar uma detecção válida ou não e os labels de quais objetos podem ser detectados.

Vale notar que o que permite a publicação de mensagens MQTT para um tópico no IoT Core da AWS é a biblioteca *awsiotsdk*, que fornece o cliente IPC para publicação sem precisar gerenciar certificados no código (o Nucleus faz isso via *token exchange*). A operação usada será *PublishToIoTCore* [15].

Por fim, o *deploy* do componente é realizado, de tal forma que toda a sequência de processos definida na *recipe* é inicializada no dispositivo Greengrass *core*.

D. Script *python*

Para o roteiro fornecido no link da Seção III, a estrutura de arquivos deve ser montada da seguinte forma no WSL:

```
artifacts/
├── vehicle-detector/1.0.0/
│   ├── main.py
│   ├── requirements.txt
│   └── models/
│       └── yolov8n.pt
```

Onde o arquivo yolov8n.pt contém os pesos e modelo do detector de objetos YOLOv8.

A Figura 11 exibe um fluxograma do *script* python responsável por realizar a inferência dos veículos via YOLOv8 das imagens capturadas pelas câmeras e realizar a publicação para o serviço de nuvem Greengrass via tópico MQTT. O *script* é instalado e executado a partir do componente implantado pela *recipe* configurada anteriormente, na Seção III-C no WSL em Windows.

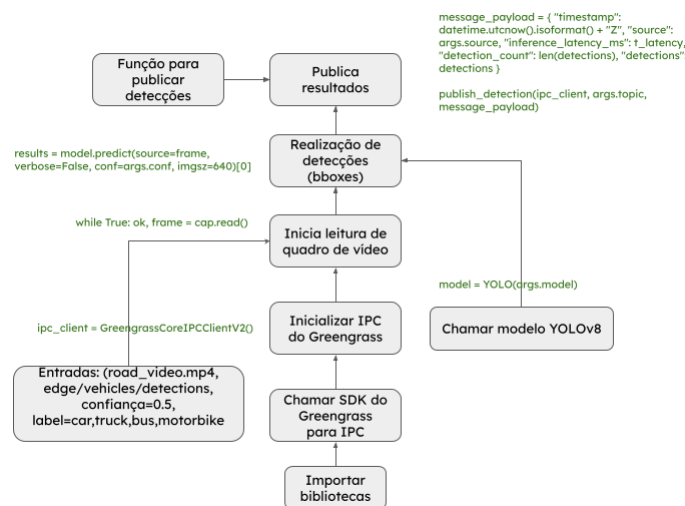


Fig. 11. Fluxograma do *script* python que realiza a inferência e publicação dos resultados obtidos pelo detector de objetos num tópico MQTT no AWS IoT. Fonte: Autor.

Conforme a Figura 11, após importar as bibliotecas descritas na Seção III-C, o SDK do Greengrass para publicação MQTT via IPC é chamado e iniciado.

Em seguida, após usar como entrada os parâmetros descritos na configuração padrão da Seção III-C, a leitura de cada quadro é realizada utilizando a biblioteca *opencv*. Após chamar o modelo YOLOv8, é realizada a inferência dos veículos a partir do cálculo das *bouding boxes* [12].

Os resultados são publicados via tópico MQTT configurado previamente utilizando uma função customizada dentro do *script*. O vídeo é executado em *loop*, portanto as publicações são feitas de maneira recorrente.

IV. RESULTADOS E DISCUSSÕES

Após a realização das etapas descritas na Seção III, foi possível configurar o notebook Dell Inspiron 5458 como dispositivo Greengrass *core*, conforme mostra a Figura 12, que

exibe a *Thing* "MyWSL_border_YOLO" criada e registrada no serviço de nuvem AWS Greengrass.

Dispositivos de núcleo do Greengrass (1)			
Um dispositivo de núcleo do AWS IoT Greengrass é um dispositivo que executa o software Greengrass Core. Depois que seu dispositivo for provisionado, ele aparecerá aqui.			
Pesquisar dispositivos de núcleo			
Nome	Status	Tempo de execução	Status relatado
MyAWS_border_VOLO	Integrado	aws_mqtt_client	há 3 minutos

Fig. 12. Captura de tela após a criação da *Thing* e configuração do instalador do *software* Greengrass no dispositivo Greengrass *core*. Fonte: Autor.

Em relação a publicação das inferências realizadas pelo *script* python rodando o YOLOv8, o *script* foi configurado para enviar as seguintes informações para o tópico "edge/vehicles/detections" via MQTT:

- **timestamp:** Instante de tempo em que a inferência foi realizada.
- Tempo de inferência do algoritmo para cada imagem (em ms);
- Número de detecções realizadas.
- Classe do veículo detectado, conforme descrito na Seção III-C.
- As *bouding boxes* de cada objeto detectado por quadro de vídeo.

A Figura 13 exibe uma captura de tela do console AWS em que podemos ver os tópicos em que o Serviço AWS Greengrass associado a conta do usuário está inscrito.

Assinaturas

Favorites

edge/vehicles/#

X

Todas as assinaturas

edge/vehicles/#

Você não pode publicar mensagens em um tópico curinga.

Selecione outro tópico para publicar mensagens.

▼ edge/vehicles/detections

```
{
  "timestamp": "2025-09-06T18:27:49.357000Z",
  "source": "/greengrass/v2/packages/artifacts/com.example.VehicleDetector/1.0.0/road_video.mp4",
  "inference_latency_ms": 168,
  "detection_count": 5,
  "detections": [
    {
      "class": "car",
      "confidence": 0.821,
      "bbox": [
        3974,
        275,
        1203,
        400
      ]
    },
    {
      "class": "car",
      "confidence": 0.802,
      "bbox": [
        0,
        162,
        71,
        257
      ]
    },
    {
      "class": "car"
    }
  ]
}
```

Fig. 13. Captura de tela do tópico MQTT "edge/vehicle/detections" recebendo publicações do dispositivo Greengrass *core* após realização do *deploy* e atuação do algoritmo YOLOv8. Fonte: Autor.

Conforme mostra a imagem, os dados descritos anteriormente são publicados pelo dispositivo Greengrass *core* no tópico MQTT configurado via cliente IPC em intervalos de milissegundos, permitindo rastreamento de veículos no trânsito de forma instantânea.

Utilizando a informação enviada pelo dispositivo de borda a nuvem AWS, outro serviço pode utilizar esses dados para monitorar veículos em situações irregulares extraindo dados das placas dos veículos a partir dos dados enviados pelo dispositivo Greengrass ou monitorando possíveis acidentes de trânsito/veículos parados na estrada quando, por exemplo, um veículo em uma *bounding box* específica é reportado de forma sequencial para a nuvem AWS, indicando que ele pode estar parado ou capotado na estrada.

A Figura 14 exibe um gráfico que relaciona o tempo de inferência (em milissegundos) durante um período de aproximadamente 4 segundos realizado pelo YOLOv8 no vídeo utilizada como fonte de dados para o detector de objetos.

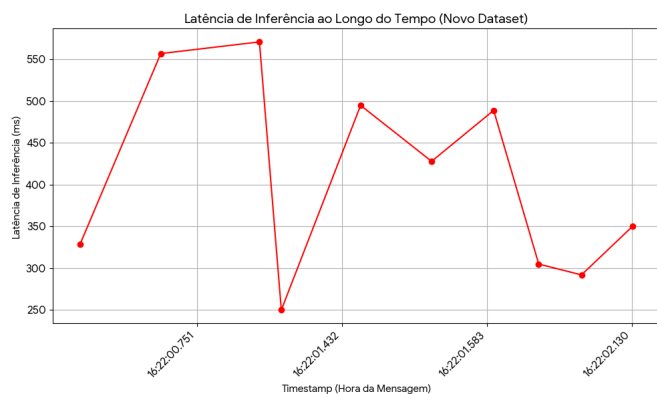


Fig. 14. Tempo de inferência do algoritmo YOLOv8 na realização de detecções numa amostragem de 4 segundos. Fonte: Autor.

Dentre outros aspectos, o tempo de inferência depende da capacidade computacional do dispositivo Greengrass *core*, o número de objetos em cada quadro de imagem, a qualidade da imagem de entrada no detector YOLOv8, entre outros.

Conforme a Figura 14, o pico de tempo para realizar a inferência dos veículos em um quadro de vídeo foi em torno de 570ms, enquanto o valor mais baixo foi de 250ms. Vale ressaltar que em outros momentos, alcançou-se valores menores que 150ms no tempo de inferência.

Devido a característica do vídeo utilizado, o número de objetos detectados pelo algoritmo de acordo com os *labels* configurados no componente foi de 9 a 17, dependendo do quadro analisado.

Não foi possível correlacionar uma maior quantidade de objetos ao tempo de inferência maior, já que em muitos casos, quadros com 17 detecções tinham tempo de inferência menor do que quadros com quantidade menor de detecções. O peso no tempo de inferência deve-se então ao uso de recursos no *host*, principalmente da CPU, que variava entre 60% a 75% durante o processamento dos quadros de vídeo.

Em ambientes ideais, o dispositivo Greengrass *core* deve ter a maior parte do seu poder de processamento dedicado para a realização da tarefa definida pelo componente, evitando esse tipo de variação acentuada.

Além disso, internamente, pelos logs do componente, foi possível visualizar a quantidade de quadros processados, a taxa de detecção do algoritmo YOLOv8 e o tamanho de cada mensagem (em bytes) enviada para o serviço de nuvem AWS Greengrass via MQTT.

De um total de 30.363 quadros devido as múltiplas instâncias em que o vídeo foi executado, a taxa de detecção foi de 100%, isto é, todos os veículos que apareciam nos quadros do vídeo de entrada foram detectados com sucesso. Isso demonstra a alta acurácia do algoritmo YOLOv8.

Obviamente, o cenário utilizado nesta solução era ideal e, na prática, câmeras com baixa resolução, quadros com interferência ou borrados por conta do ambiente ou condições climáticas serão enviados para o detector de objetos, impondo um desafio adicional para o YOLOv8.

Finalmente, a Figura 15 exibe um gráfico que relaciona o tamanho do *payload* MQTT enviada para o serviço de nuvem Greengrass em bytes em relação ao tempo.

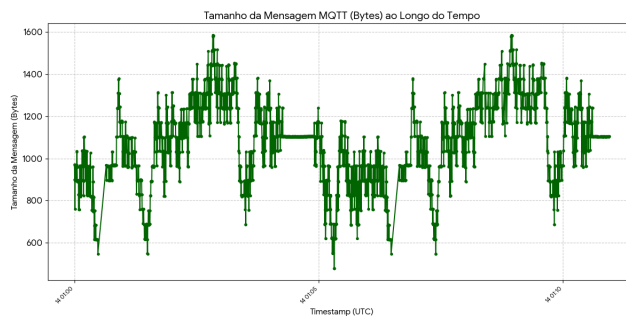


Fig. 15. Tamanho do *payload* MQTT enviado pelo *script* python para o AWS. Fonte: Autor.

De acordo com a Figura 15, pode-se notar que há uma variação no tamanho das mensagens entre 500 Bytes até valores próximos de 1600 Bytes. Essa variação deve-se, principalmente, a quantidade de detecções diferentes entre quadros, tal que para quadros com maior quantidade de detecções, mais informação deve ser enviada e, por conta disso, o tamanho do *payload* MQTT aumenta.

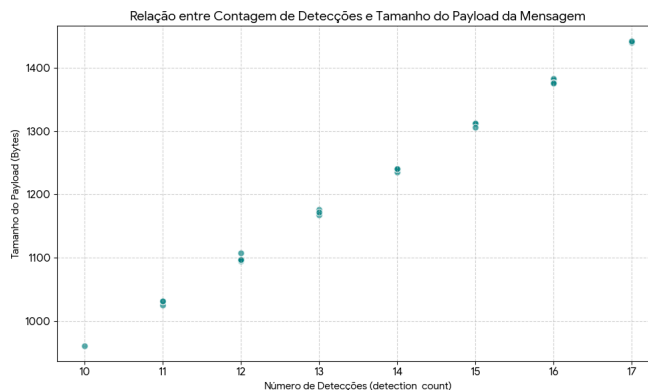


Fig. 16. Gráfico de dispersão entre o tamanho do *payload* MQTT e a quantidade de detecções por quadro. Fonte: Autor.

Isso fica claro ao avaliar o gráfico da Figura 16, que exibe um gráfico de dispersão entre o tamanho do *payload* MQTT e a quantidade de detecções por quadro.

Conforme a figura, na medida que o número de detecções num determinado quadro aumenta, o tamanho da mensagem MQTT tende a aumentar de maneira relativamente linear.

Todavia, vale notar que mesmo para mensagens maiores (de 1600 Bytes, por exemplo), o custo na largura de banda necessária para transmitir a informação de inferência para o serviço de nuvem AWS é muito menor do que enviar um quadro de vídeo inteiro que, mesmo em resoluções menores (como HD), tem tamanhos acima de 500 KB. Isso viabiliza soluções que necessitam de resposta em tempo real, já que o transporte de dados não se torna o gargalo.

V. CONCLUSÃO E PERSPECTIVAS

Este trabalho apresentou a implementação e validação de uma solução de computação em borda para detecção de veículos, utilizando o AWS Greengrass V2 e o modelo YOLOv8. O objetivo principal de demonstrar a viabilidade de processamento local para Sistemas de Transporte Inteligente (ITS) foi alcançado com sucesso.

Os resultados demonstram que a abordagem de computação em borda é altamente eficaz na superação das limitações dos modelos tradicionais de computação em nuvem, particularmente em aplicações críticas que demandam baixa latência e eficiência no uso da rede.

As principais conclusões e contribuições do trabalho são:

- **Validação da Baixa Latência:** A latência de inferência do YOLOv8, rodando no dispositivo Greengrass core (Notebook Dell Inspiron 5458), variou entre 250 ms e 570 ms, com a capacidade de atingir valores inferiores a 150 ms em outros momentos. Essa faixa de tempo é crucial para o processamento em tempo real exigido pelos ITSs.
- **Eficiência de Banda Comprovada:** As mensagens MQTT publicadas na AWS contendo os resultados da inferência foram consistentemente menores que 1.6 KB. Este tamanho de *payload* é significativamente menor do que a transmissão de quadros de vídeo brutos (que excedem 500 KB, mesmo em baixa resolução), eliminando o gargalo da largura de banda.
- **Correlação Detecção vs. Tamanho:** Foi confirmada uma correlação positiva e linear entre o número de detecções em um quadro e o tamanho do *payload* MQTT, indicando que o tamanho da mensagem é diretamente proporcional à informação relevante extraída.
- **Integração e Arquitetura:** O projeto validou a integração completa das tecnologias, utilizando o AWS Greengrass V2 para gerenciamento e *deploy* de componentes, o protocolo MQTT para comunicação segura com o AWS IoT Core e o YOLOv8 para a tarefa de ML na borda.

A. Perspectivas Futuras

Para futuros trabalhos, sugerimos as seguintes direções para aprimorar a solução:

- 1) **Otimização de Hardware:** Realizar o *deploy* do Greengrass core em um dispositivo de borda dedicado (como um Raspberry Pi ou hardware industrial) para eliminar a variabilidade de latência causada pela contenção de recursos do sistema operacional hospedeiro, que atualmente causa picos de uso de CPU de 60% a 75%.
- 2) **Robustez em Cenários Reais:** Testar o desempenho do YOLOv8 com vídeos de câmeras de vigilância de rua com baixa resolução ou condições climáticas adversas (*e.g.*, chuva, neblina, pouca luz) para avaliar a acurácia e a taxa de detecção do algoritmo em ambientes não ideais.

Em resumo, este trabalho serve como um forte ponto de partida para a implementação de soluções de inteligência artificial na borda, fornecendo uma base robusta e de baixo custo para o desenvolvimento de sistemas ITS escaláveis.

REFERENCES

- [1] E. Yurtsever, J. Lambert, A. Carballo and K. Takeda, "A Survey of Autonomous Driving: Common Practices and Emerging Technologies," in *IEEE Access*, vol. 8, pp. 58443-58469, 2020, doi: 10.1109/ACCESS.2020.2983149.
- [2] J. Zhang, F. -Y. Wang, K. Wang, W. -H. Lin, X. Xu and C. Chen, "Data-Driven Intelligent Transportation Systems: A Survey," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 4, pp. 1624-1639, Dec. 2011, doi: 10.1109/TITS.2011.2158001.
- [3] M. Swarnamugi and R. Chinnaiyan, "IoT Hybrid Computing Model for Intelligent Transportation System (ITS)," 2018 Second International Conference on Computing Methodologies and Communication (ICCMC), Erode, India, 2018, pp. 802-806, doi: 10.1109/ICCMC.2018.8487843.
- [4] W. Shi and S. Dustdar, "The Promise of Edge Computing," in *Computer*, vol. 49, no. 5, pp. 78-81, May 2016, doi: 10.1109/MC.2016.145.
- [5] MAO, Y. et al. A Survey on Mobile Edge Computing: The Communication Perspective. *IEEE Communications Surveys & Tutorials*, v. 19, n. 4, p. 2322-2358, 2017. DOI: 10.1109/COMST.2017.2745201.
- [6] Amazon Web Services. "Como o AWS IoT funciona," AWS IoT Core Developer Guide, [Online]. Disponível em: https://docs.aws.amazon.com/pt_br/iot/latest/developerguide/aws-iot-how-it-works.html. [Acesso: Set. 27, 2025].
- [7] Amazon Web Services. "Como o AWS IoT funciona," AWS IoT Core Developer Guide, [Online]. Disponível em: https://docs.aws.amazon.com/pt_br/iot/latest/developerguide/aws-iot-how-it-works.html. [Acesso: Set. 27, 2025].
- [8] Amazon Web Services. "O que é AWS IoT Greengrass?," AWS IoT Greengrass Versão 2 Guia do Desenvolvedor, [Online]. Disponível em: <https://docs.aws.amazon.com/greengrass/v2/developerguide/what-is-iot-greengrass.html>. [Acesso: Set. 27, 2025].
- [9] Amazon Web Services. "Como funciona o AWS IoT Greengrass," AWS IoT Greengrass Versão 2 Guia do Desenvolvedor, [Online]. Disponível em: <https://docs.aws.amazon.com/greengrass/v2/developerguide/how-it-works.html>. [Acesso: Set. 27, 2025].
- [10] ZHANG, C.; PATRAS, P.; HADDADI, H. Deep Learning in Mobile and Wireless Networking: A Survey. *IEEE Communications Surveys & Tutorials*, v. 21, n. 3, p. 2224-2287, 2019. DOI: 10.1109/COMST.2019.2904897.
- [11] HATCHER, W. G.; YU, W. A Survey of Deep Learning: Platforms, Applications and Emerging Research Trends. *IEEE Access*, v. 6, p. 24411-24432, 2018. DOI: 10.1109/ACCESS.2018.2830661
- [12] AMJOUR, A. B.; AMROUCH, M. Object Detection Using Deep Learning, CNNs and Vision Transformers: A Review. *IEEE Access*, v. 11, p. 35479-35516, 2023. DOI: 10.1109/ACCESS.2023.3266093.

- [13] SINGH, G.; TIWARI, S.; SINGH, J. Real Time Object Detection using Neural Networks: A Comprehensive Survey. In: 2023 Third International Conference on Artificial Intelligence and Smart Energy (ICAIS). [S.l.: s.n.], 2023. P. 1281–1286. DOI: 10.1109/ICAIS56108.2023.10073826.
- [14] Reis, D., Kupec, J., Hong, J., & Daoudi, A. (2023). Real-time flying object detection with YOLOv8. arXiv preprint arXiv:2305.09972.
- [15] Matheus Alves, "Solução de Inteligência na Borda com AWS Greengrass V2 para Reconhecimento de Veículos usando YOLO," GitHub, [Online]. Disponível em: <https://github.com/maalvess/olu-o-de-Intelig-ncia-na-Borda-com-AWS-Greengrass-V2-para-Reconhecimento-de-Ve-culos-usando-YOLO>. [Acesso: Set. 27, 2025].
- [16] Ultralytics Inc. "Explore Ultralytics YOLOv8," Documentação Ultralytics YOLO, [Online]. Disponível em: <https://docs.ultralytics.com/pt/models/yolov8/>. [Acesso: Set. 27, 2025].